Analizador Sintáctico

Rocío Corral[1]

[1]Lic. Cs. de la Computación/Facultad de Ingeniería/UNCuyo

9 de Abril de 2023

Abstract

Proceso de construcción de un analizador léxico para el lenguaje de programación TinyRust+. Proceso de transformación de la gramática y construcción de un analizador sintáctico descendente predictivo recursivo. Decisiones de diseño, metodología, lenguaje utilizado, arquitectura, funcionamiento y resultados se explican en el presente trabajo.

1. Introducción

El objetivo de este informe es explicar y fundamentar la implementación de las primeras dos etapas del compilador de TinyRust+.

En la primera etapa se implementó el analizador léxico, programa que genera un conjunto de *tokens* a partir de un archivo de texto. Estos tokens se definen utilizando como base las reglas descritas en el manual de TinyRust+ para delimitar palabras reservadas, operadores, identificadores, tipos y demás.

En la segunda etapa se implementó el analizador sintáctico, programa que consume *tokens* generados por el analizador léxico y verifica que el programa cumpla con las reglas de la gramática de TinyRust+. Esta implementación consiste en un analizador sintáctico descendente predictivo recursivo. Para su implementación, se tuvo que tratar la gramática de TinyRust+ descrita en el manual transformándola a notación BNF, eliminando la recursividad a izquierda y factorizando.

2. Metodología

Para el desarrollo de este proyecto, el primer paso fue hacer un análisis de la información disponible sobre el funcionamiento esperado de TinyRust+ y definir una serie de aspectos de diseño detallados en la siguiente sección. Posterior a esto, se diagramó la arquitectura del proyecto y finalmente se implementó secuencialmente cada etapa.

A la hora de la implementación, se trabajó con el lenguaje de programación Java. Se utilizó Git junto con GitHub como herramienta de control de versiones, de manera tal que ambos integrantes del equipo pudieran trabajar colaborativa e independientemente en el proyecto de manera simultánea.

1.1 Consideraciones

A continuación se listan una serie de consideraciones y decisiones de diseño relevantes

antes de profundizar en la implementación del compilador.

El alfabeto de entrada del compilador consta de los caracteres con código ASCII desde 32 hasta 126[1]. Adicionalmente, también se incluyen los caracteres:

- Salto de línea (ASCII 10)
- Tabulación (ASCII 9)
- Fin de texto (ASCII 3)
- Tabulación vertical (ASCII 11)

En la Tabla 1 se encuentra definido por extensión el alfabeto de TinyRust+.

[space]	!	"	#	$	%	&	'	(	)	*	+	,	-	.	0	1	2	3	4
[tab]	5	6	7	8	9	:	;	<	=	>	?	@	A	B	C	D	E	F	G
[vtab]	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
[break]	[	]	_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
[etx]	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	\	/	^	

Tabla 1. Alfabeto de entrada de TinyRust+.

Cabe destacar que ciertos caracteres exclusivos del español, como “ñ” y vocales con tilde (tanto mayúsculas como minúsculas) no forman parte del alfabeto de entrada de este compilador.

En las tablas 2 y 3 se puede apreciar la lista de tokens que el analizador léxico reconoce. En cada tabla se encuentra el tipo del token acompañado de la expresión regular que lo representa. La Tabla 2 muestra la lista de tokens de palabras reservadas del lenguaje, mientras que la Tabla 3 contiene todos los demás tokens reconocidos tales como literales, operadores, comentarios y delimitadores.

Tipo de token	Expresión regular
p_class	class
p_else	else
p_false	false
p_if	if
p_return	return
p_while	while
p_true	true

Tipo de token	Expresión regular
p_nil	nil
p_false	false
p_new	new
p_func	fn
p_static	static
p_pub	pub
p_void	void

Tabla 2. Tokens de palabras reservadas.

Tipo de token	Expresión Regular	Notas
id	(a..z + _)(a..z + A..Z + _ + 0..9)*	Identificador: comienzan con una letra minúscula o guión bajo y luego pueden contener cualquier cantidad de letras minúsculas, mayúsculas, números o guiones bajos. Exceptuando las palabras reservadas.
lit_int	$(0..9)^+$	Literal entero: cualquier sucesión de dígitos numéricos.
lit_string	“Σ*”	Literal cadena: contiene cualquier cantidad de caracteres del alfabeto de entrada encerrados entre comillas dobles.
lit_char	‘(Σ + \Σ)’	Literal caracter: contiene posiblemente un backslash y luego un único caracter del alfabeto de entrada encerrado entre comillas simples.
type	(A..Z)(a..z + A..Z + _ + 0..9)*	Tipo de dato: similar a un identificador, solo que comienza con una letra mayúscula.
multi_line_comment	/* Σ* */	Comentario multilínea: puede contener cualquier cantidad y tipo de caracteres (incluyendo saltos de línea) encerrados entre ambas secuencias de apertura y cierre.

single_line_comment	//Σ* ([break] + [etx])	Comentario de una línea: comienza con un par de *slashes* y puede contener cualquier cantidad y tipo de caracteres, finalizando en un salto de línea (o final del archivo).
p_true	true	
p_false	false	
op_add	+	
op_sub	-	
op_prod	*	
op_div	/	
op_mod	%	
op_or	||	
op_and	&&	
op_less	<	
op_great	>	
op_less_eq	<=	
op_great_eq	>=	
op_equal	==	
assignment	=	
semicolon	;	
colon	:	
open_par	(	
close_par	)	
open_curly	{	

close_curly	}	
open_bracket	[	
close_bracket	]	
return_type	->	
op_not	!	
op_not_eq	!=	
comma	,	
dot	.	

Tabla 3. Tokens de literales, operadores, comentarios y delimitadores.

2.2 Errores

En la etapa de diseño se previó que una serie de errores distintos pueden aparecer durante cada etapa del análisis. Por este motivo, los tipos de errores están clasificados según la etapa de análisis en la que aparezcan.

Los posibles errores previstos en la etapa del análisis léxico son:

- *Identificador mal formado*: error generado al encontrar un lexema que es cercano a un identificador pero no cumple con su sintaxis por algún motivo. Por ejemplo, la cadena '123var' comienza con dígitos seguidos de letras, por lo cual no es ni un literal entero ni un identificador. Otro ejemplo podría ser una cadena que comienza con minúsculas y, sin un espacio de por medio, tiene un "@", por ejemplo, "vari@ble".
- *Comentario sin cerrar*: error generado al encontrar una secuencia de apertura de comentario multilínea y luego llegar al final del archivo sin encontrar la secuencia de cierre, es decir, "*/".
- *Símbolo no perteneciente al alfabeto*: error generado al leer en el archivo de entrada un caracter que no esté incluido en el alfabeto Σ, como por ejemplo: "¬", "ñ", "á", etc.
- *Token desconocido*: error generado al encontrar un lexema que no cumple ningún patrón reconocible, por ejemplo, "|". Cabe destacar que "||" sí es reconocido como el operador lógico OR. Sin embargo, "|" por sí solo, no es un token de este compilador.
- *Literal inválido*: error generado al hallar literales de tipo cadena o caracter con una estructura inválida. Por ejemplo, literal cadena sin cerrar o literal caracter conteniendo más de un caracter.

Para la etapa de análisis sintáctico se previó un único error *Token inesperado*, generado cuando se encuentra un token que no coincide con ninguna regla gramatical de TinyRust+, teniendo en cuenta la secuencia previa de tokens del archivo.

2.1 Transformación de la gramática

La cátedra facilitó en el manual de TinyRust+ una versión de su gramática en formato EBNF. Para hacer uso de la gramática para la implementación del analizador sintáctico descendente predictivo recursivo, se aplicaron una serie de algoritmos para transformar la gramática, sin alterar el conjunto de cadenas generados por la gramática. Los algoritmos aplicados fueron: transformación a formato BNF, eliminación de reglas lambda, eliminación de recursividad a izquierda y factorización. Finalmente, se calcularon el conjunto de primeros y siguientes para los símbolos no terminales de la gramática. En las siguientes secciones se explican de forma detallada las modificaciones realizadas a cada paso de la transformación.

2.1.1 **BNF**

El primer paso fue transformar a notación BNF la gramática de TinyRust+ proporcionada por la cátedra en formato EBNF. Se utilizó el algoritmo proporcionado por la cátedra para transformar las notaciones de tipo opcional (‘?’) y repetición (‘*’) a su notación equivalente en BNF. Se listan a continuación únicamente las reglas de la gramática que contenían dicha notación exclusiva de EBNF.

```
Start ::= Clase* Main
Clase ::= "class" "idClase" Herencia? "{" Miembro* "}"
Atributo ::= Visibilidad? Tipo ":" Lista-Declaración-Variables ";"
Método ::= Forma-Metodo? "fn" "idMétodoVariable" Argumentos-Formales "->"
    Tipo-Método Bloque-Método
Argumentos-Formales ::= "(" Lista-Argumentos-Formales? ")"
Bloque-Método ::= "{" Decl-Var-Locales* Sentencia* "}"

Sentencia ::= ";"
    | Asignacion";"
    | Sentencia-Simple ";"
    | "if" "(" Expresion ")" Sentencia
    | "if" "(" Expresion ")" Sentencia "else" Sentencia
    | "while" "(" Expresion ")" Sentencia
    | Bloque
    | "return" Expresion?";"

Bloque ::= "{" Sentencia* "}"

Asignacion-Variable-Simple  ::= "id" Encadenado-Simple* | "id" "[" Expresion "]"
Asignacion-Self-Simple ::= "self" Encadenado-Simple*

Operando ::= Literal | Primario Encadenado?
ExpresionParentizada ::= "(" Expresion ")" Encadenado?
AccesoSelf ::= "self" Encadenado?
AccesoVar ::= "id" Encadenado?
Llamada-Método ::= "id" ArgsActuales  Encadenado?
Llamada-Método-Estático ::= "idClase" "." Llamada-Metodo Encadenado?
Llamada-Constructor ::= "new" "idClase" Argumentos-Actuales Encadenado? | "new"
```

```
    Tipo-Primitivo "[" Expresion "]"

Argumentos-Actuales ::= "(" Lista-Expresiones? ")"
Llamada-Método-Encadenado ::= "id" Argumentos-Actuales Encadenado?
Acceso-Variable-Encadenado ::= "id" Encadenado? | id "[" Expresion "]"
```

Este subconjunto de reglas que no cumplían con las restricciones de la notación BNF se reemplazó con el siguiente conjunto de reglas equivalentes que sí cumplen con la notación.

```
Start ::= Clases Main
Clases ::= Clase Clases | λ
Clase ::= "class" "idClase" HerenciaOp "{" Miembros "}"
Miembros ::= Miembro Miembros | λ
HerenciaOp ::= Herencia | λ
Atributo ::= VisibilidadOp Tipo ":" Lista-Declaración-Variables ";"
VisibilidadOp ::= Visibilidad | λ

Método ::= Forma-MetodoOp "fn" "idMétodoVariable"  Argumentos-Formales "->"
Tipo-Metodo Bloque-Metodo
Forma-MétodoOp ::= Forma-Metodo | λ

Argumentos-Formales ::= "(" Lista-Argumentos-FormalesOp ")"
Lista-Argumentos-FormalesOp ::= Lista-Argumentos-Formales | λ

Bloque-Método ::= "{" Decl-Var-LocalesN Sentencias "}"
Decl-Var-LocalesN := Decl-Var-Locales Decl-Var-LocalesN | λ
Sentencias ::= Sentencia Sentencias | λ

Sentencia ::= ";"
      | Asignacion";"
      | Sentencia-Simple ";"
      | "if" "(" Expresion ")" Sentencia
      | "if" "(" Expresion ")" Sentencia "else" Sentencia
      | "while" "(" Expresion ")" Sentencia
      | Bloque
      | "return" ExpresionOp";"

ExpresionOp := Expresion | λ
Bloque ::= "{" Sentencias "}"

Asignacion-Variable-Simple  ::= "id" Encadenado-SimpleN | "id" "[" Expresion "]"

Encadenado-SimpleN ::= Encadenado-Simple Encadenado-SimpleN | λ

Asignacion-Self-Simple ::= "self" Encadenado-SimpleN

EncadenadoOp ::= Encadenado | λ
Operando ::= Literal | Primario EncadenadoOp
ExpresionParentizada ::= "(" Expresion ")" EncadenadoOp
AccesoSelf ::= "self" EncadenadoOp
```

```
AccesoVar ::= "id" EncadenadoOp
Llamada-Método ::= "id" ArgsActuales  EncadenadoOp
Llamada-Método-Estático ::= "idClase" "." Llamada-Metodo EncadenadoOp
Llamada-Constructor ::= "new" "idClase" Argumentos-Actuales EncadenadoOp |
"new" Tipo-Primitivo "[" Expresion "]"

Argumentos-Actuales ::= "(" ExpresionesOp ")"
ExpresionesOp ::= Lista-Expresiones | λ
Llamada-Método-Encadenado ::= "id" Argumentos-Actuales EncadenadoOp
Acceso-Variable-Encadenado ::= "id" EncadenadoOp | id "[" Expresion "]"
```

2.1.2 Eliminación de producciones lambda

A partir de la gramática en notación BNF, se procedió a eliminar algunas reglas lambda antes de eliminar la recursividad por la izquierda. En este caso no se eliminaron todas las producciones lambda, dado que algunas de estas reglas resultan luego en una implementación más práctica del analizador sintáctico y no generan problemas durante la eliminación de recursividad a izquierda. Las reglas que se eliminaron durante este proceso fueron las siguientes:

```
Clase ::= "class" "idClase" HerenciaOp "{" Miembros "}"
HerenciaOp ::= Herencia | λ
Atributo ::= VisibilidadOp Tipo ":" Lista-Declaración-Variables ";"
VisibilidadOp ::= Visibilidad | λ

Método ::= Forma-MetodoOp "fn" "idMétodoVariable"  Argumentos-Formales "->"
Tipo-Metodo Bloque-Metodo
Forma-MétodoOp ::= Forma-Metodo | λ

Sentencia ::= ";"
      | Asignacion";"
      | Sentencia-Simple ";"
      | "if" "(" Expresion ")" Sentencia
      | "if" "(" Expresion ")" Sentencia "else" Sentencia
      | "while" "(" Expresion ")" Sentencia
      | Bloque
      | "return" ExpresionOp ";"

ExpresionOp := Expresion | λ
```

A continuación se listan las nuevas reglas que se agregaron para reemplazar a las anteriores:

```
Clase ::= "class" "idClase" "{" Miembros "}"
      | "class" "idClase" Herencia "{" Miembros "}"

Atributo ::= Tipo ":" Lista-Declaración-Variables ";"
      | Visibilidad Tipo ":" Lista-Declaración-Variables ";"

Método ::= Forma-Método "fn" "idMétodoVariable" Argumentos-Formales "->"
Tipo-Método Bloque-Método
```

```
        | "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método
    Bloque-Método

    Sentencia ::= ";"
        | Asignación";"
        | Sentencia-Simple ";"
        | "if" "(" Expresion ")" Sentencia
        | "if" "(" Expresion ")" Sentencia "else" Sentencia
        | "while" "(" Expresion ")" Sentencia
        | Bloque
        | "return" ";"
        | "return" Expresión ";"
```

2.1.3 **Eliminación de la recursividad por la izquierda**

A partir de la gramática obtenida en el paso anterior, se procedió a eliminar la recursividad por la izquierda. Este procedimiento se realizó de manera ordenada en sucesivos pasos, como indica la guía proporcionada por la cátedra. El algoritmo indica que se deben modificar las reglas cuyo primer símbolo coincide con su parte derecha o con la parte derecha de cualquier regla previa. Se descubrió que en el orden dado de las reglas, no se requería modificar ninguna regla del segundo tipo (coincidencia con la parte derecha de una regla previa). Se remplazaron únicamente las reglas que contenían recursividad inmediata por la izquierda. Se lista a continuación el conjunto de reglas que se eliminaron en este paso:

```
ExpOr ::= ExpOr "||" ExpAnd | ExpAnd
ExpAnd ::= ExpAnd "&&" ExpIgual | ExpIgual
ExpIgual ::= ExpIgual OpIgual ExpCompuesta | ExpCompuesta
ExpAdd ::= ExpAdd OpAdd ExpMul | ExpMul
ExpMul ::= ExpMul OpMul ExpUn | ExpUn
```

Ese conjunto de reglas fue reemplazado por las siguientes reglas, obtenidas mediante el algoritmo de eliminación de recursividad inmediata por la izquierda:

```
ExpOr ::= ExpAnd ExpOrP
ExpOrP ::= "||" ExpAnd ExpOrP | λ
ExpAnd ::= ExpIgual ExpAndP
ExpAndP ::= "&&" ExpIgual ExpAndP | λ
ExpIgual ::= ExpCompuesta ExpIgualP
ExpIgualP ::= OpIgual ExpCompuesta ExpIgualP | λ
ExpAdd ::= ExpMul ExpAddP
ExpAddP ::= OpAdd ExpMul ExpAddP | λ
ExpMul ::= ExpUn ExpMulP
ExpMulP ::= OpMul ExpUn ExpMulP | λ
```

2.1.4 **Factorización**

Como último paso, se factorizó la gramática obtenida tras eliminar la recursividad a la izquierda. No se factorizó exhaustivamente la gramática, dado que complicaría de más la etapa de implementación del analizador. Se eligieron a criterio las reglas a factorizar

de modo que faciliten la implementación y aseguren una derivación recursiva determinista para toda regla. Las reglas modificadas fueron las siguientes:

```
Clase ::= "class" "idClase" "{" Miembros "}"
        | "class" "idClase" Herencia "{" Miembros "}"

Sentencia ::= ";"
        | Asignación";"
        | Sentencia-Simple ";"
        | "if" "(" Expresion ")" Sentencia
        | "if" "(" Expresion ")" Sentencia "else" Sentencia
        | "while" "(" Expresion ")" Sentencia
        | Bloque
        | "return" ";"
        | "return" Expresión ";"
```

Al factorizar estas dos producciones, se obtuvieron las siguientes nuevas reglas en su lugar:

```
Clase ::= "class" "idClase" ClaseHerenciaOp
ClaseHerenciaOp ::= Herencia "{" Miembros "}"  | "{" Miembros "}"

Sentencia ::= ";"
        | Asignación ";"
        | Sentencia-Simple ";"
        | "if" "(" Expresión ")" Sentencia ElseOp
        | "while" "(" Expresión ")" Sentencia
        | Bloque
        | "return" Expresión ";"
        | "return" ";"

ElseOp ::= "else" Sentencia | λ
```

2.1.5 **Gramática final**

Se encuentra a continuación la gramática final completa y ordenada obtenida tras aplicar todos los algoritmos explicados previamente. Esta gramática fue la que se utilizó posteriormente para implementar el analizador sintáctico descendente predictivo recursivo.

```
Start ::= Clases Main
Clases ::= Clase Clases | λ

Main ::=  "fn" "main" "(" ")" Bloque-Método
Miembros ::= Miembro Miembros | λ

Clase ::= "class" "idClase" ClaseHerenciaOp
ClaseHerenciaOp ::= Herencia "{" Miembros "}"  | "{" Miembros "}"

Herencia ::= ":" "idClase"
Miembro ::= Atributo | Constructor | Método
```

```
Atributo ::= Tipo ":" Lista-Declaración-Variables ";"
    | Visibilidad Tipo ":" Lista-Declaración-Variables ";"

Constructor ::= "create" Argumentos-Formales Bloque-Método
Método ::= "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método
Bloque-Método
    | Forma-Método "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método
Bloque-Método

Argumentos-Formales ::= "(" ")"     | "(" Lista-Argumentos-Formales ")"

Lista-Argumentos-Formales ::= Argumento-Formal "," Lista-Argumentos-Formales |
Argumento-Formal
Argumento-Formal ::= Tipo ":" "idMétodoVariable"
Forma-Método ::= "static"
Visibilidad ::= "pub"
Tipo-Método ::= Tipo | "void"
Tipo ::= Tipo-Primitivo | Tipo-Referencia | Tipo-Array
Tipo-Primitivo ::= "Bool" | "I32" | "Str" | "Char"
Tipo-Referencia ::= "idClase"
Tipo-Array ::= "Array" Tipo-Primitivo
Lista-Declaración-Variables ::= "idMétodoVariable" | "idMétodoVariable" ","
Lista-Declaración-Variables
Bloque-Método ::= "{" Decl-Var-LocalesN Sentencias "}"

Decl-Var-LocalesN := Decl-Var-Locales Decl-Var-LocalesN | λ

Decl-Var-Locales ::= Tipo ":" Lista-Declaración-Variables ";"

Sentencias ::= Sentencia Sentencias | λ

Sentencia ::= ";"
      | Asignación ";"
      | Sentencia-Simple ";"
      | "if" "(" Expresión ")" Sentencia ElseOp
      | "while" "(" Expresión ")" Sentencia
      | Bloque
      | "return" Expresión ";"
      | "return" ";"

ElseOp ::= "else" Sentencia | λ

Bloque ::= "{" Sentencias "}"

Asignación ::= Asignación-Variable-Simple "=" Expresión | Asignación-Self-Simple
"=" Expresión
Asignación-Variable-Simple  ::= "idMétodoVariable" "[" Expresión "]" |
"idMétodoVariable" Encadenado-SimpleN

Encadenado-SimpleN ::= Encadenado-Simple Encadenado-SimpleN | λ
```

Asignación-Self-Simple ::= "self" Encadenado-SimpleN

Encadenado-Simple ::= "." "idMétodoVariable"

Sentencia-Simple ::= "(" Expresión ")"

Lista-Expresiones ::= Expresión | Expresión "," Lista-Expresiones

Expresión ::= ExpOr
ExpOr ::= ExpAnd ExpOrP
ExpOrP ::= "||" ExpAnd ExpOrP | λ
ExpAnd ::= ExpIgual ExpAndP
ExpAndP ::= "&&" ExpIgual ExpAndP | λ
ExpIgual ::= ExpCompuesta ExpIgualP
ExpIgualP ::= OpIgual ExpCompuesta ExpIgualP | λ
ExpCompuesta ::= ExpAdd OpCompuesto ExpAdd | ExpAdd
ExpAdd ::= ExpMul ExpAddP
ExpAddP ::= OpAdd ExpMul ExpAddP | λ
ExpMul ::= ExpUn ExpMulP
ExpMulP ::= OpMul ExpUn ExpMulP | λ
ExpUn ::= OpUnario ExpUn | Operando

OpIgual ::= "==" | "!="
OpCompuesto ::= "<" | ">" | "<=" | ">="
OpAdd ::= "+" | "-"
OpUnario ::= "+" | "-" | "!"
OpMul ::= "*" | "/" | "%"
Operando ::= Literal | Primario EncadenadoOp
Literal ::= "nil" | "true" | "false" | "intLiteral" | "stringLiteral" | "charLiteral"
Primario ::= ExpresionParentizada | AccesoSelf | AccesoVar | Llamada-Método |
Llamada-Método-Estático | Llamada-Constructor
ExpresionParentizada ::= "(" Expresión ")" EncadenadoOp
AccesoSelf ::= "self" EncadenadoOp
AccesoVar ::= "idMétodoVariable" EncadenadoOp | "idMétodoVariable" "["
Expresión "]"
Llamada-Método ::= "idMétodoVariable" Argumentos-Actuales EncadenadoOp
Llamada-Método-Estático ::= "idClase" "." Llamada-Método EncadenadoOp
Llamada-Constructor ::= "new" "idClase" Argumentos-Actuales EncadenadoOp
| "new" Tipo-Primitivo "[" Expresión "]"

Argumentos-Actuales ::= "(" ")" | "(" Lista-Expresiones ")"
Encadenado ::= "." Llamada-Método-Encadenado | "." Acceso-Variable-Encadenado

Llamada-Método-Encadenado ::= "idMétodoVariable" Argumentos-Actuales
EncadenadoOp
Acceso-Variable-Encadenado ::= "idMétodoVariable" EncadenadoOp |
"idMétodoVariable" "[" Expresión "]"
EncadenadoOp ::= Encadenado | λ

2.2 **Primeros y siguientes**

2.2.1 **Primeros**

A partir de la versión final de la gramática, se procedió a calcular la lista de primeros y siguientes para cada símbolo no terminal de la gramática. La lista obtenida es la siguiente:

Start = {"class", "fn"}
Clases = {"class", λ}
Clase = {"class"}
Main = {"fn"}
Miembros = {"Bool", "I32", "Str", "Char", "idClase", "Array", "pub", "create", "fn", "static", λ}
Herencia = {":"}
ClaseHerenciaOp = {":", "{"}
Miembro = {"Bool", "I32", "Str", "Char", "idClase", "Array", "pub", "create", "fn", "static"}
Atributo = {"Bool", "I32", "Str", "Char", "idClase", "Array", "pub"}

Constructor = {"create"}
Método = {"fn", "static"}
Argumentos-Formales = {"("}
Lista-Argumentos-Formales = {"Bool", "I32", "Str", "Char", "idClase", "Array"}
Argumento-Formal = {"Bool", "I32", "Str", "Char", "idClase", "Array"}
Forma-Método = {"static"}
Visibilidad = {"pub"}
Tipo-Método = {"Bool", "I32", "Str", "Char", "idClase", "Array", "void"}
Tipo = {"Bool", "I32", "Str", "Char", "idClase", "Array"}
Tipo-Primitivo = {"Bool", "I32", "Str", "Char"}
Tipo-Referencia = {"idClase"}
Tipo-Array = {"Array"}
Lista-Declaración-Variables = {"idMétodoVariable"}
Bloque-Método = {"{"}

Decl-Var-LocalesN = {"Bool", "I32", "Str", "Char", "idClase", "Array", λ }
Decl-Var-Locales = {"Bool", "I32", "Str", "Char", "idClase", "Array"}

Sentencias = {";", "idMétodoVariable", "self", "(", "if", "while", "{", "return", λ}
Sentencia = {";", "idMétodoVariable", "self", "(", "if", "while", "{", "return"}
ElseOp = {"else", λ}
Bloque = {"{"}

Asignación = {"idMétodoVariable", "self"}
Asignación-Variable-Simple = {"idMétodoVariable"}

Encadenado-SimpleN = {".", λ}
Asignación-Self-Simple = {"self"}

Encadenado-Simple = {"."}
Sentencia-Simple = {"("}

Lista-Expresiones = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}

Expresión = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}

ExpOr = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpOrP = {"||", λ}

ExpAnd = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpAndP = {"&&", λ}

ExpIgual = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpIgualP = {"==", "!=", λ}

ExpCompuesta = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}

ExpAdd = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpAddP = {"+", "-", λ}

ExpMul = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpMulP = {"*", "/", "%", λ}

ExpUn = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}

OpIgual = {"==", "!="}
OpCompuesto = {"<", ">", "<=", ">="}
OpAdd = {"+", "-"}
OpUnario = {"+", "-", "!"}
OpMul = {"*", "/", "%"}
Operando = {"nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new" }
Literal = {"nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral"}
Primario = {"(", "self", "idMétodoVariable" , "idClase", "new"}

ExpresionParentizada = {"("}
AccesoSelf = {"self"}
AccesoVar = {"idMétodoVariable"}
Llamada-Método = {"idMétodoVariable"}
Llamada-Método-Estático = {"idClase"}
Llamada-Constructor = {"new"}

Argumentos-Actuales = {"("}
Encadenado = {"."}
Llamada-Método-Encadenado = {"idMétodoVariable"}
Acceso-Variable-Encadenado = {"idMétodoVariable"}
EncadenadoOp = {".", λ}

2.2.2 **Siguientes**

El cálculo de siguientes se realizó únicamente para los símbolos cuyos primeros incluyen lambda. Esta decisión es con fines de practicidad, ya que durante la implementación del analizador sintáctico de un lenguaje LL1, se requerirá a lo sumo saber los siguientes de los símbolos que derivan lambda. Posteriormente la implementación se pudo lograr sin hacer uso en lo absoluto de este conjunto de siguientes, pero el cálculo se realizó de todos modos y el conjunto de siguientes para las reglas que derivan lambda se encuentra a continuación.

Clases = {"fn"}
Miembros = {"}"}
Sentencias = {"}"}
ElseOp = {"else", ";", "idMétodoVariable", "self", "(", "if", "while", "{", "return", "}"}
Decl-Var-LocalesN = {";", "idMétodoVariable", "self", "(", "if", "while", "{", "return", "}"}
Encadenado-SimpleN = {"="}
ExpOrP = {";", ")", "]", ","}
ExpAndP = {"||", ";", ")", "]", ","}
ExpIgualP = {"&&", "||", ";", ")", "]", ","}
ExpAddP = {"<", ">", "<=", ">=", "==", "!=", "&&", "||", ";", ")", "]", ","}
ExpMulP = { "+", "-", "<", ">", "<=", ">=", "==", "!=", "&&", "||", ";", ")", "]", ","}
EncadenadoOp = {".", "*", "/", "%", "+", "-", "<", ">", "<=", ">=", "==", "!=", "&&", "||", ";", ")", "]", ","}

3. Resultados

3.1 **Estructura general**

El repositorio del proyecto se encuentra estructurado en una jerarquía de directorios para facilitar su legibilidad. En la Figura 1 puede apreciarse esta jerarquía.

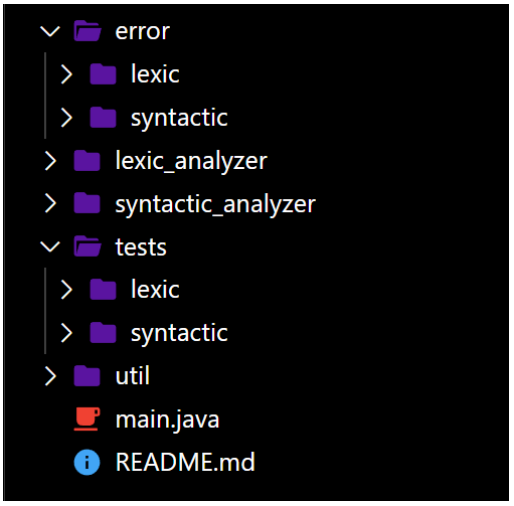


Figura 1. Jerarquía de carpetas

En el directorio raíz se encuentran los siguientes archivos y directorios:

4. Archivo *main.java*: contiene la clase TinyRustCompiler, el punto de entrada del compilador.

5. Carpeta *util*: contiene clases que proveen algún tipo de asistencia mediante métodos. Por ejemplo: la clase Logger, que provee métodos necesarios para escribir mensajes en el archivo de salida.

6. Carpeta *tests*: contiene los archivos *.rs* de prueba utilizados para probar el compilador. Contiene una carpeta por cada etapa de análisis con archivos de prueba para esa etapa. Cada archivo de prueba incluye el output esperado al analizarlo.

7. Carpeta *lexic_analyzer:* contiene todas las clases relacionadas al analizador léxico del compilador. Incluye al analizador léxico, un ejecutor y las clases *Token* y *ReservedWords*.

8. Carpeta *syntactic_analyzer*: contiene todas las clases relacionadas al analizador sintáctico del compilador. Incluye al analizador sintáctico y un ejecutor.

9. Carpeta *error*: contiene una clase para cada posible error que pueda ser arrojado al momento de ejecutar el compilador de TinyRust+. Contiene una carpeta por cada etapa de análisis, donde se encuentra una clase para cada error que pueda aparecer en esa etapa.

9.1 **Implementación**

En esta sección se ahondará en la implementación del analizador sintáctico de TinyRust+, explicando el rol, funcionamiento y métodos de cada una de las clases que lo componen. La Figura 2 muestra el diagrama de clases UML del proyecto completo.

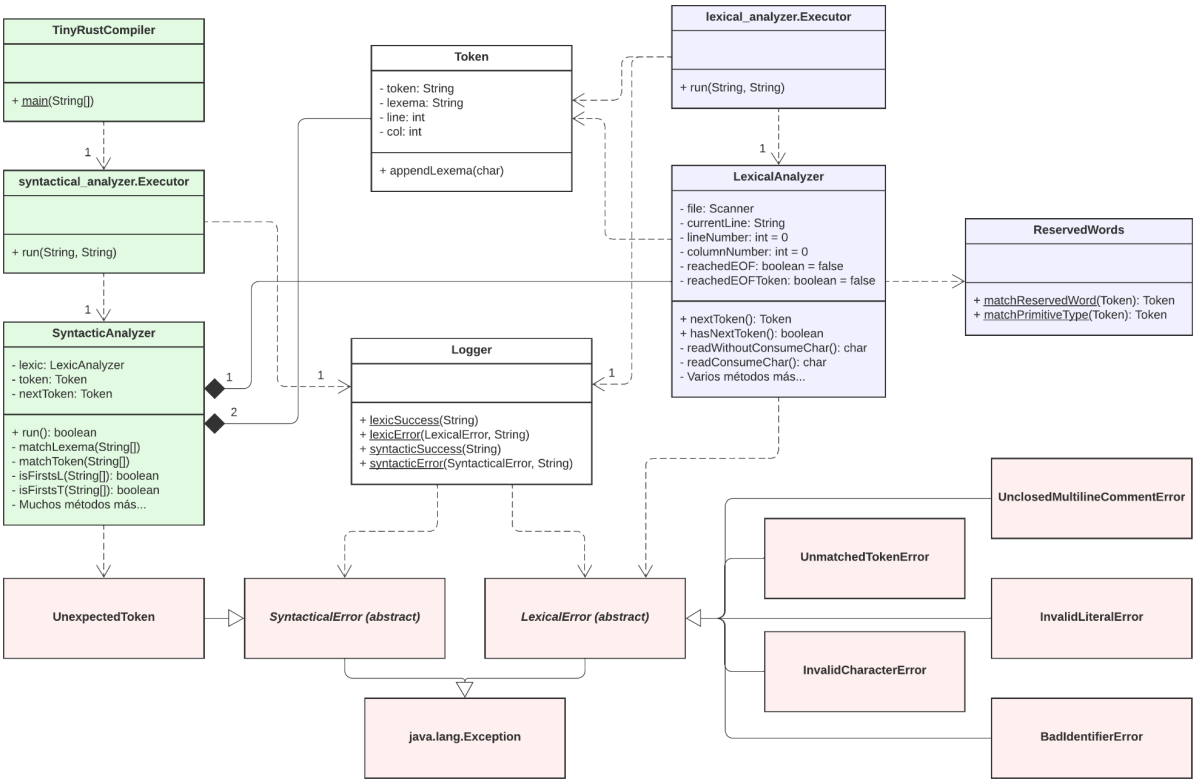


Figura 2. Diagrama de clases UML del compilador de TinyRust+.

9.1.1 **TinyRustCompiler**

Esta es la clase raíz del compilador. Recibe los argumentos de la línea de comandos para los archivos de entrada y salida y se encarga de guardarlos. Además, en el método main de esta clase se instancia la clase *Executor* y se invoca su método *run* para comenzar la ejecución del compilador.

9.1.2 **Executor (lexic_analyzer/Executor)**

Esta clase consta permite poner a prueba el analizador léxico de forma aislada. Consta de un único método llamado *run*, que crea una instancia de la clase LexicAnalizer y le pasa como parámetro la ruta del archivo de entrada. Una vez instanciado el analizador léxico, el ejecutor llamará iterativamente al método *nextToken* del *LexicAnalyzer*, el cual devuelve el siguiente token encontrado en el archivo de entrada. Al recibir el token, lo agrega a una lista de *Token,* la cual será mostrada en el archivo de salida o por pantalla en el caso de tener un análisis léxico exitoso. Caso contrario, la salida mostrará en qué línea, columna y token se presentó el error.

9.1.3 **LexicAnalizer**

Esta clase es la encargada de todo el proceso de análisis léxico. En su constructor se abre el archivo de entrada e inicializan contadores de línea y columna.

LexicAnalizer cuenta con diversos métodos que permiten el correcto análisis léxico de un archivo de texto. Provee únicamente dos métodos públicos, que se describen a continuación:

nextToken: consume parte del archivo de entrada para reconocer y retornar el siguiente token. En caso de haber consumido todo el archivo, devuelve el token EOF (end of file). Si ocurriera algún error al intentar decodificar el archivo, este método arrojaría un error del tipo *LexicalError.*

hasNextToken: verifica si ya se ha alcanzado el final del archivo y devuelve un valor booleano acorde. Este método puede invocarse como validación antes de invocar a *nextToken*. Siempre que *hasNextToken* devuelva *false*, está garantizado que *nextToken* devolverá el token EOF (end of file).

A su vez, *LexicAnalyzer* implementa una gran cantidad de métodos privados que son utilizados directa o indirectamente por el método *nextToken*. Los métodos privados de la clase se listan y describen a continuación.

Validadores: *isValidChar, isLowercaseChar, isUppercaseChar, isAlphabet, isDigit, isOperator, isBlankSpace, isLineBreak.* Estos métodos reciben una variable de tipo char y verifican si ese caracter pertenece a cierto conjunto en particular.

Matchers: *matchIntLiteral*, *matchStringLiteral, matchCharLiteral, matchTypeIdentifier, matchMultilineComment, matchComment.* Estos métodos reciben una variable de tipo char y una variable de tipo *Token*. A partir del caracter inicial recibido, verifican si lo que le sigue en el archivo de entrada coincide con lo que se espera para un token en particular. Devuelven un booleano indicando si se pudo construir por completo el token esperado y actualizan el contenido del token recibido como parámetro. Pueden arrojar excepciones en algunos casos de token mal formado o caracteres inválidos.

consumeSpaces: se encarga de descartar espacios en blanco del archivo de entrada. Arroja *InvalidCharacterError* si se encuentra con algún caracter no perteneciente al alfabeto de entrada

readConsumeChar: lee el siguiente caracter del archivo, lo consume y aumenta el contador de columna. Es decir, una vez leído no hay vuelta atrás. Si se encuentra con el fin de una línea, verifica si hay una línea más. De ser así, incrementa el contador de línea, reinicia el contador de columna y continúa leyendo la siguiente línea del archivo. Si encuentra el final del archivo, marca que se llegó al final de la lectura y cierra el archivo.

readWithoutConsumeChar: análogo a *readConsumeChar*, lee el siguiente caracter del archivo pero no lo consume. Es decir, no se pierde el caracter leído. Si no se invoca a la función *readConsumeChar*, la función *readWithoutConsumeChar* devuelve siempre el mismo caracter en llamados sucesivos.

9.1.4 **Executor (syntactic_analyzer/Executor)**

Esta clase consta de un único método llamado *run*. Al invocar este método, se crea una instancia de la clase SyntacticAnalyzer que recibe como parámetro la ruta del archivo de entrada. Una vez instanciado el analizador, el ejecutor llamará al método *run* del SyntacticcAnalizer, el cual devuelve un booleano indicando si el archivo de entrada

es un archivo sintácticamente válido de TinyRust+. En caso de un resultado positivo, un mensaje de éxito es escrito en el archivo de salida o mostrado por pantalla. Si el archivo no fuera sintácticamente correcto, esta clase intercepta la excepción lanzada por SyntacticAnalyzer y en la salida muestra fila, columna y token donde se presentó el error, incluyendo en la descripción si se trata de un error léxico o sintáctico.

9.1.5 **SyntacticAnalyzer**

Esta clase es la encargada de todo el proceso de análisis sintáctico. El análisis se hace de forma descendente y predictiva, mientras que la implementación en esta clase es recursiva.

En su constructor se inicializa una instancia de LexicAnalyzer pasándole la ruta del archivo de entrada.

SyntacticAnalyzer provee un único método público, *run*. Este método comienza el proceso de análisis sintáctico del archivo utilizando la gramática de TinyRust+. Durante el análisis sintáctico, se irán solicitando y consumiendo tokens del analizador léxico y se verificará que el archivo de entrada sea derivable a partir del símbolo inicial de la gramática, llamando recursivamente métodos de la clase que implementan cada regla de la gramática.

La clase tiene dos atributos privados relevantes: *token* y *nextToken*. Ambos son instancias de la clase Token y son generados por el analizador léxico. Ambas variables son inicializadas con los primeros dos tokens del archivo al ejecutar el método *run.* Luego su valor va cambiando a medida que el análisis sintáctico avanza y los tokens son *matcheados* en reglas de la gramática. Cada vez que se *matchea* y consume un token, se mueve el valor de *nextToken* a *token* y se solicita un nuevo token al analizador léxico para actualizar el nextToken. En caso de que el analizador léxico arroje una excepción, el analizador sintáctico no la atrapa, sino que la propaga hacia arriba al siguiente invocador.

A su vez, SyntacticAnalyzer posee una enorme cantidad de métodos privados, algunos de ellos se describen a continuación:

matchToken: compara el tipo del token actual contra la lista de tipos de token recibida. Si coincide con algún elemento de la lista, consume el token. Caso contrario, lanza una excepción de tipo *UnexpectedToken*.

matchLexema: compara el lexema del token actual contra la lista de lexemas recibida. Si coincide con algún elemento de la lista, consume el token. Caso contrario, lanza una excepción de tipo *UnexpectedToken*.

isFirstT: compara el tipo del token actual contra la lista de tipos de token recibida y retorna un booleano indicando si coincide con algún elemento de la lista. La principal diferencia con *matchToken* es que *isFirstT* lee el token sin consumirlo.

isFirstL: compara el lexema del token actual contra la lista de lexemas recibida y retorna

un booleano indicando si coincide con algún elemento de la lista. La principal diferencia con *matchLexema* es que *isFirstL* lee el token sin consumirlo.

Otros métodos: el resto de métodos privados de la clase *SyntacticAnalyzer* se corresponden cada uno con la regla homónima de la gramática final de TinyRust+. Cada método implementa la lógica de la regla correspondiente. El método ejecutará por completo sin problemas si la secuencia de tokens encontrados es derivable mediante esa regla de la gramática, caso contrario arrojará una excepción sin completar su ejecución.

9.2 Errores

Para el manejo de errores de cada etapa de análisis se creó una clase abstracta distinta que hereda de la clase Exception de Java. Por ejemplo, los errores lanzados por el analizador léxico heredan de la clase abstracta *LexicalError*, mientras que los errores del analizador sintáctico heredan de *SyntacticalError*. Una nueva instancia de cualquiera de estas clases de excepción espera recibir el número de línea y columna donde ocurrió el error junto con una descripción. A la hora de mostrar el mensaje de error, este incluirá estos tres datos.

En el caso del análisis léxico, para cada tipo de error distinto que pueda ocurrir durante el análisis se creó una clase distinta que herede de *LexicalError*. Los distintos tipos de error posibles en esta etapa se explican en la sección 2.2.

En el caso del análisis sintáctico, todos los errores lanzados son de tipo *UnexpectedToken*, que hereda de la clase *SyntacticalError*. Esta excepción se produce cuando se encuentra un token que no coincide con ninguna regla gramatical de TinyRust+, teniendo en cuenta la secuencia previa de tokens. Este error incluye en cada caso un mensaje especificando el lexema que se encontró y el o los posibles tokens/lexemas que se esperaba encontrar en ese contexto. En ocasiones este error incluye una descripción de la estructura gramatical que se esperaba en vez de incluir la especificación de cada posible token esperado. Por ejemplo, cuando se espera una expresión y se recibe un token que no podría ser el primer token de ninguna expresión válida, el mensaje dirá “se esperaba una expresión, se encontró [token]”. Esto es debido a que un mensaje listando el conjunto de 14 tokens válidos para comenzar una expresión podría no resultar de ayuda para el usuario que intenta compilar un archivo de TinyRust+.

9.3 Casos de prueba

En el código del proyecto se incluyen los casos de prueba que se utilizaron para poner a prueba el funcionamiento de cada etapa de análisis. Dichos archivos se pueden encontrar en el directorio *tests* del proyecto, donde se encuentra una carpeta por cada etapa de análisis, conteniendo archivos de prueba con casos exitosos y casos de fallo.

La lista de tests consiste en 30 archivos de prueba para el análisis léxico y más de 100

archivos de prueba para el análisis sintáctico. Se detalla a continuación la estructura de los casos de prueba para la etapa de análisis sintáctico.

Particularmente el archivo de test número 0 corresponde al ejemplo de código de TinyRust+ provisto por la cátedra en el manual del lenguaje. El análisis sintáctico de este archivo es exitoso.

Los primeros cuatro archivos de prueba, sumados a unos pocos archivos extra entre los 100 casos de prueba, contienen casos exitosos. Estos archivos contienen una gran cantidad de código sintácticamente válido de TinyRust+ para poner a prueba que casos de uso comunes y casos borde sean reconocidos correctamente.

Mientras que el resto de archivos de prueba (la gran mayoría) son casos de falla, en los que se espera que el compilador arroje cierto error en un token en particular que no respete la gramática del lenguaje. Cada archivo incluye al final un comentario especificando en qué lugar del archivo se encontró un token inesperado y cuál es el o los tokens esperados en ese lugar del archivo, dado el contenido previo.

9.4 Compilado y ejecución

Para compilar el proyecto, deben ejecutarse los siguientes comandos desde el directorio raíz del proyecto.

Paso 1: compilar.

```
javac error/*.java lexic_analyzer/*.java syntactic_analyzer/*.java util/*.java *.java
```

Paso 2: generar el jar.

```
jar cfm etapa2.jar Manifest.txt *
```

Para ejecutar el proyecto debe ejecutarse el siguiente comando:

```
java -jar etapa2.jar <ARCHIVO_FUENTE> [<ARCHIVO_SALIDA>]
```

Es obligatorio especificar la ruta del archivo fuente que TinyRust+ analizará. En el directorio *tests* se pueden hallar archivos de prueba para alimentar el comando. Por ejemplo:

```
java -jar etapa2.jar ./tests/syntactic/"test (3).rs" ./output-example.o
```

Especificar un archivo de salida es opcional. En caso de ser especificado, se escribirá la salida del compilador en dicho archivo. Si el archivo existe previamente se escribirá la salida al final del archivo, caso contrario será creado por el compilador. En caso de no especificarse este parámetro, la salida se mostrará por pantalla.

10. Conclusiones

Ya finalizando la segunda etapa del desarrollo del compilador de TinyRust+, el proyecto comienza a tomar forma. El analizador léxico implementado en la etapa previa resultó encastrar a la perfección con el analizador sintáctico y los archivos analizados ya empiezan a parecer código real de TinyRust+.

Resulta interesante que el trabajo más costoso de esta etapa fue el tratado de la gramática, mientras que la implementación de este tipo de analizador sintáctico no fue compleja, sino más bien metódica. La desventaja de esto es que el proceso es muy propenso a errores humanos y el código es difícil de modularizar y de debuggear.

11. Referencias

1. ASCII-CODE. ASCII table according to Windows-1252. https://www.ascii-code.com/